ASSIGNMENT: NEXT WORD PREDICTION USING LANGUAGE MODELING

Course: Advanced Natural Language Processing

Topic: Language Modeling & Next Word Prediction

OBJECTIVE

The objective of this assignment is to help students understand and implement **Language Models (LMs)** in NLP by building a system that predicts the **next word in a sequence** based on given input text.

Students will learn:

- NLP pipeline design
- Text preprocessing
- N-gram / statistical language modeling
- Probability-based prediction
- Model evaluation

PROBLEM STATEMENT

Design and implement a **Next Word Prediction System** using NLP techniques.

The system should take a sentence (or partial sentence) as input and predict the **most probable next word**.

LEARNING OUTCOMES

After completing this assignment, students will be able to:

- Understand the concept of Language Modeling
- Implement N-gram models (Unigram, Bigram, Trigram)
- Apply probability-based prediction
- Build an end-to-end NLP pipeline

- Evaluate model performance

TASK REQUIREMENTS

Task 1: Dataset Selection

Choose a dataset from any of the following:

- News articles
- Wikipedia text
- Movie dialogues
- Custom text corpus

Minimum requirement:

- At least 5000 sentences

Task 2: NLP Pipeline Implementation

Students must implement the complete NLP pipeline:

1. Text Cleaning
    - Convert to lowercase
    - Remove punctuation
    - Remove special characters
2. Tokenization
    - Split text into words
3. Stopword Handling (Optional but recommended)
    - Remove common stopwords OR justify why not removed
4. Vocabulary Creation
    - Create a list of unique words

Task 3: Language Model Implementation

Students must implement:

- Unigram Model
  - Probability of a word independently
- Bigram Model
  - Probability based on previous word
- Trigram Model
  - Probability based on previous two words

Task 4: Probability Calculation

Students must compute:

- Word probabilities
- Conditional probabilities

Examples:

- $P(word_2 \mid word_1)$
- $P(word_3 \mid word_1, word_2)$

Task 5: Next Word Prediction System

Build a function:

predict_next_word(input_text)

Example:

- Input: "the cat"
- Output: "eats" (or top 3 predictions)

Task 6: Smoothing Techniques

Implement at least one smoothing method:

- Laplace Smoothing (Add-1)

- Good-Turing (optional advanced)

Explain why smoothing is needed.

Task 7: Evaluation

Evaluate your model using:

- Accuracy (if test data available)
- Perplexity (recommended for advanced students)

Task 8: Comparison

Compare:

- Unigram vs Bigram vs Trigram

Discuss:

- Which performs better and why

IMPLEMENTATION GUIDELINES

- Programming Language: Python
- Libraries (optional):
    - NLTK
    - spaCy
    - NumPy
    - Pandas

Important:

- Do not use pre-built next-word prediction APIs
- Focus on manual implementation of logic

DELIVERABLES

Students must submit:

1. Code File
    - Well-structured and commented
2. Report (5–10 pages)
   Include:
    - Introduction to language modeling
    - Dataset description
    - NLP pipeline explanation
    - Model implementation
    - Results and observations
    - Challenges faced
3. Demo (Optional but recommended)
    - Show predictions for sample inputs

SAMPLE INPUTS FOR TESTING

Students must test on examples like:

- "the cat is"
- "machine learning is"
- "natural language processing"
- "the boy went to"

BONUS TASKS (For Extra Marks)

- Implement Top-K predictions (top 3 or 5 words)
- Add GUI (Streamlit / Flask)
- Use Neural Language Model (LSTM / Transformer)
- Compare statistical vs neural approaches